Server notes and other indexes

Server notes and other indexes

This part contains 21 notes, from `ARCHIVAL.md` through `server-notes/README.md`.

Server notes (except the setcharprop note, which is in the NPC volume), Ash notes, earlier archival notes, and the consolidated summary.

`ARCHIVAL.md`

Archival notes — local research workspace decommissioned

Date: 2026-10-28

The local Graal research workspace (gserver dumps, Phoenix binary package, Ash Python rewrite, Ember architecture atlas, and the 39-chapter GServerStory narrative) has been decommissioned.

The full contents were archived as a single zip:

- **Google Drive:** `/[Grok/[Projects/Graal gServers.zip`
- **Direct link:** (Drive link omitted from the document release)
- **File ID:** `(omitted from the document release)`
- **Size:** ~27 MB

That archive contains the original source dumps (`graal gservers/`), the Reborn 2.4.0 Phoenix package, the Ash Python reference server, the Ember atlas (markdown + ATLAS.html), the complete GServerStory, and supporting files.

What is already in this repository

This repo (`Graal_analysis`) holds the cleaned client-focused notes, protocol tables, format documentation, version profiles, and decompile findings derived from public sources and observed behavior. It deliberately excludes raw decompile material.

What was added from the local workspace

Under `server-notes/` you will find high-value extracts from the Ember atlas and related materials that document the *server* side of the same protocol surface:

- Lineage of the dumps (1.3.x monolith → gs2emu → Phoenix → Ash)
- Protocol framing, G-integers, login, props, file send, listserver IDs
- Algorithms and pseudocode (LCG, gen-3 dummy byte, gen-4/5 XOR stream, login, warp, baddies, trigger hacks, RC file browser, etc.)
- Architecture and on-disk blueprints (see the original Ember files in the Drive archive for the full set)

These notes are faithful to the dumps and the Ash rewrite; nothing was invented about the protocol. They complement the client-centric material already in this repository.

Ash as a readable reference

The Ash Python server (included in the Drive archive) is a deliberate cut-down rewrite of the gs2emu protocol surface. It implements gens 3–5, GRACC001 accounts, GLEVNW01 levels, listserver register/ping, RC, gmaps, baddies, and trigger-action hacks. It is useful as a readable companion to the C++ dumps when implementing a server.

Prior attempts

The local workspace also contained earlier attempts to map the environment:

- Phoenix: packaged Reborn 2.4.0 binary (port 14802) with a named world
- Ash: the Python rewrite described above (port 14812, offline login)
- Ember: the architecture atlas that walks the dumps
- GServerStory: a narrative walk through the engines in learning order (chapters 1–39 + command index)

All of those materials remain available in the Drive zip for anyone who needs the original trees or the full narrative.

This file was added when the local research project was decommissioned. The Drive archive is the authoritative source for the original dumps and the complete Ember / GServerStory materials.

`archival/client-assets-measured.md`

Client asset tree (measured)

Date: 2026-10-14 (archived 2026-10-28)

Measured from the official client install. No decompile material.

Tile sheets

File	Size	Role
pics1.png	2048×512	Standard tiles, 128×32 of 16 px, 4096 ids
picso.png	1024×512	Offline tileset (type 1)
sprites.png	128×452	Sprite sheet
letters.png	—	Bitmap font

Tile id `t` maps to `col = t % 128`, `row = t // 128`, source rect `(col*16, row*16, 16, 16)` on pics1.

Tiledef

Offline line recovered from the tree:

```
picso.png,offlinetutorial,1,0,0
```

Format is `image,prefix,type[,x,y]`. Type 0 is the old Graal order (pics1). Type 1 is the new order.

levels/ counts (files on disk)

Dir	Files	Contents
ganis/	93	86 .gani plus images
midis/	49	music
heads/	35	33 png, 2 mng
weapons/	33	weapon images, 2 txt
bomys/	29	bomy images and ganis
emoticons/	29	png and mng
webgifs/	26	mostly png
backpals/	23	background palettes
bodies/	20	body images
images/	19	misc images
smilies/	15	
baddies/	10	one sheet per baddy
shields/	7	
swords/	6	
npcs/	5	script text
horses/	4	
hats/	4	
tutorial/	4	
letters/	2	blueletters, graal2002letters
snapshot/	2	ss_arrow, ss_bomb
tiledefs/	1	offline tiledef line

Later work in `client-internals/client-assets.md` and the version-profile notes supersedes this snapshot where counts or mappings differ.

`archival/foot-test-sdl.md`

Foot test (SDL reimplementation)

Date: 2026-10-14 (archived 2026-10-28)

Derived from the SDL reimplementation of the client movement loop and the type table. This is **not** the real client's walking test.

The real clients (2.05 through 2.31) use a six-probe body-box routine with corner nudges and flush snaps. See Seihai `documentation/walking-collision.md` for the current description.

Single-point test (SDL)

Arrow keys set a pixel step of `16 * 0.30` and a sprite direction (0 up, 1 left, 2 down, 3 right). The wall test is a single point:

```
testx = (pixelX + moveX + 32) / 16
testy = (pixelY + moveY + 44) / 16
```

If `isOnWall(testx, testy)` is false, the new position is accepted. If true, the step is rejected. There is no slide, no corner resolution, and no water slowdown in this loop.

Predicates used by the SDL path

- Outside the board on x or y: wall.
- Inside: type byte at `levelTiles[round(y)*64 + round(x)]` ≥ 20 is wall; type == 11 is water (does not block).
- Chair, bed, swamp, and lava types are stored and ignored by this path.

The 2×2 Bush stamp writes ids 2, 3, 18, 19. Those type bytes are 20/22, so the bush is wall under this path.

`archival/README.md`

Archival notes

Date: 2026-10-28

Prior attempts and early reconstructions of the Graal environment, preserved for reference. These notes predate the later decompile-finding batches and the Seihai walking-collision recovery. They remain useful as a record of the measurement path that locked formats, stamps, and the type table.

No raw decompile material is included.

Contents

- `foot-test-sdl.md` — the single-point wall test recovered from the SDL reimplementation (2dadventureclient). The real client uses a six-probe body-box routine; see Seihai `documentation/walking-collision.md` and the later Graal_analysis client notes for the current description.
- `client-assets-measured.md` — measured sizes and counts from the official client install (pics1 2048×512, picso, levels/ subdirectory counts, offline tiledef line).
- `version-tokens-early.md` — early public token map (GNW07080 = 1.39, GNW03081 = 2.05, through GNW28015 = 2.3.1.0) and filename builds.
- `tools-early.md` — short notes on GraalEditor, GraalShop, the three generators, gif2mng, and RemoteControl as first measured from format output.

These files are snapshots. Later work in `client-internals/`, `protocol/`, and `tools/` supersedes them where they conflict.

`archival/tools-early.md`

Tools (early measurements)

Date: 2026-10-14 (archived 2026-10-28)

Short notes on the classic tool chain as first measured from format output. Later work in `tools/` and the level-editor notes supersedes these where they conflict.

GraalEditor

Version 2.3.1.1, Linux Cyberjoueur. From 2.3 the editor and the online client are separate executables. The copy shipped in the client tree is a Thinstall package. Main form `TAdventure`, document `TAdventureLevel`. Writes

decimal `BOARD 0` grids (64 lines of 64 indices) plus LINK/SIGN/CHEST/NPC/BADDY. A file of raw decimal indices does not pass the client `GLEVNW01` loader until rewritten as base64 runs.

GraalShop

Delphi GANI editor. Sprite mode (`GANI0001`) and movie mode (`GANI0002`). Actor kinds CHAR, SPRITE, SOUND, TEXT.

Generators

Three writers emit `GLEVNW01` / `GRMAP001`. They do not emit the same file.

- TerrainGenerator: plasma midpoint displacement → `GRMAP001` with `HEIGHTS` blocks (9×9) and `GLEVNW01` levels that are 64 rows of tile 0 plus a `HEIGHTS` overlay.
- DungeonGenerator: 8×11 stamp layout → one base64 `BOARD` of 64×64, edge `LINK`s, and `NPC` blocks. Stamp id 0 is a 4×4 ground fill; stamp id 1 is a hole and is not written.
- GraalLevelGen: image-to-linked-levels pipeline.

gif2mng

GIF to PNG or MNG. Banner date 2004-06-03. `-b` snaps the palette to the seven Graal body slots and forces transparent index 2.

RemoteControl

Delphi staff client, predecessor of the GTK `RC2_2.05`. Two sockets (RC at console +0x38c, NPC at +0x390). Classic text codec (id + 0x20). NPC login is type 3 starting with the literal `NCL11012`. Player list packet 0x37 carries a property stream.

`archival/version-tokens-early.md`

Version tokens (early map)

Date: 2026-10-14 (archived 2026-10-28)

Internal version strings recovered from the public version list at the time of first measurement.

Token	Build notes
GNW07080	1.39
GNW03081	2.05 (and 2002ver9)
GNW31101	2.1.0.0–2.1.0.3 (version string not updated)
GNW30042	2.1.4.0 (first with server-list tabs)
GNW15053	2.1.9.0 (bz2 + new encryption, new server-list gfx)
GNW01113	2.2.1.1 (plaintext and zlib; extra compression byte)
GNW14015	2.3.0.0
GNW28015	2.3.1.0

Filename builds in the client tree: Graal_2171 = 2.171, Graal_222 = 2.22, Graal_231 = 2.31.

From 2.3 the editor and the online client are separate executables. PE FileVersion is authoritative for the editor (2.3.1.1).

Later work in `client-internals/version-tokens.md`, `build-catalogue.md`, and the version-profile TOML files supersedes this table where tokens or mappings differ.

`ARCHIVE.md`

Archive note — Graal Analysis project

Date: 2026-10-28

The analysis project that produced these notes is being decommissioned.

What is archived

A complete zip of the local analysis corpus (`Graal Client Analysis.zip`, ≈102 MB) has been placed in Google Drive under `[Grok/[Projects/`.

It contains: - Dated snapshots (2026-10-01, 2026-10-02, 2026-10-04) - Unixsane incremental passes - Phoenix2 / Phoenix3 architecture and ledger - Lumiere, Ash, Lantern notes - Sign decoder, tiledef masks and plugin - Seihai drop-in packages (formats, protocols, version token, full v2.05 corpus) - The printable Graal v2.05 Client Reference Manual

Boundary (unchanged)

Raw decompiles, function addresses, and binary dumps stay out of this repository. The notes here remain cleaned observations derived from public sources, captures, and format documentation.

Prior attempts (summary)

The analysis corpus captured successive passes against Graal.exe v2.05 rev2 and earlier clients:

- Early field notes and architecture collapse (Phoenix)
- GS1 dispatch plating and row table
- Session / list / world protocol framing and version-token construction
- Tile-walk rules, sign alphabets, config and drop-files behaviour
- Offline training levels versus session levels
- Cross-version builds, list-host behaviour, and world rejections

Those passes are preserved in the Drive zip. The cleaned notes in this repository are the shareable subset.

Next

Seihai is the active client project. Further protocol, format, and testing work continues there. This repository remains a reference for the cleaned observations.

ash/ALGORITHMS.md

39ster 2.1 — algorithms and pseudocode

All numeric fields that go through `CBuffer::writebyte` / `readbyte` / `writebyteN` / `readbyteN` use GCHAR. Frame lengths and the raw tile blob do not.

Integer types below are the C types in the 2006 source (`int` iterator, `char` for many coords). Treat iterator as 32-bit wrapping on `*=` if you port; the original multiplies a signed `int`.

A1 — GCHAR numeric codec

Purpose: keep packet bytes in the printable range (`value + 32` → starts at ASCII space). Multi-byte integers are 7-bit big-endian groups, each biased by +32.

```
function enc1(v):               # writebyte
    emit byte (v + 32) & 0xFF

function dec1(b):               # readbyte
    return (signed char)(b - 32)

function encN(v, n):            # writebyte2/3/4/5
    for i from n-1 down to 0:
        emit byte (((v >> (7*i)) & 0x7F) + 32)

function decN(bytes, n):        # readbyte2/3/4/5
    acc = 0
    for i in 0 .. n-1:
        acc = (acc << 7) + (bytes[i] - 32)
    return acc
```

Widths actually used:

Helper	Bytes	Shift start	Used for
writebyte	1	—	opcodes, most props, coords*2
writebyte2	2	7	player id, AP counter, script length
writebyte3	3	14	rupees, kills, deaths, npc id
writebyte4	4	21	effect colors (inbound only)
writebyte5	5	28	file mtime, last-ip, mod-warp mtime

`writechar` / `readchar` / `addBuffer` write raw octets. Frame length is raw BE u16 (not GCHAR).

Strings are raw: `len = enc1(n); emit n bytes`. Head GIF is special: `enc1(100 + n)` then n bytes of filename. If inbound head length `< 100`, treat it as a classic head index and format `head{n}.gif`.

A4 — PLAYER descramble (drop-one LCG)

Iterator is initialized in `CPlayer::CPlayer` to `0x04A80B38`. `key` is the second GCHAR of the login line.

```
# parseMessage, CLIENTPLAYER only
function descramble(line, iterator, key) -> payload:
    iterator = iterator * 0x8088405 + key
    pos = ((iterator & 0xFFFF) % line.count) + 1     # 1-based
```

```
    # drop the byte at index pos-1
    return line[0 .. pos-2] + line[pos .. end]

# CLIENTRC: payload = line unchanged
```

Then `opcode = readbyte(payload)` (GCHAR) and `msgFuncs[opcode](rest)`.

A8 — GR-V1 tile bitstream

Maps are 64×64 tiles. Magic: `"GR-V1.01"` / `"GR-V1.02"` / `"GR-V1.03"`.

- 01 uses 12-bit codes; 02/03 use 13-bit.
- Bitstream decoder with run-length and double-repeat modes.

Full pseudocode and remaining algorithms (outbound/inbound zlib frames, ID generation, board-change persistence notes, etc.) are in the Drive archive `Graal 39sters gServer.zip` under `Ash/ALGORITHMS.md`.

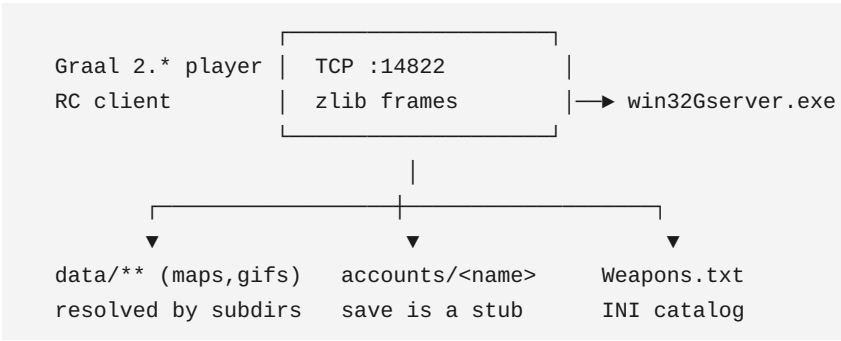
This file captures the critical wire-format and codec details needed to understand or reimplement the 2006 server behavior.

`ash/ARCHITECTURE.md`

39ster 2.1 GServer — architecture

December 2006 open remake of the 1.39 Graal server, targeting Graal clients 2.*. Single-process Windows/POSIX C++ server. No script VM. No MySQL despite the readme. No `.nw` loader despite the stub.

1. System context



External actors:

- **CLIENTPLAYER (0)** — game client. Login packet carries a scramble `key`. Every later inbound line drops one byte via LCG.
- **CLIENTRC (1)** — Remote Control client. Same framing, **no** scramble. RC opcodes 51–87 exist in the table; bodies are empty stubs.
- Filesystem only. Readme promised MySQL credentials; `loadAccount` hard-codes spawn.

2. Process and thread model

Two threads, shared heaps, no locks.

```
main thread
│
├─ listen on 14822 (non-blocking)
├─ accept loop (select 1 s)
├─ for each new connection: spawn player thread
└─ every 1 s: gameTime++, idle-kick check (300 s), weapon list reload?

player threads (one per connection)
│
├─ mainLoop: recv → inflate frames → split lines → (descramble if PLAYER) → parseMessage
└─ sendoutgoing on flush or buffer pressure
```

`maxPlayers` is 100 but never enforced. `playerIds` is a simple free-list.

3. Class graph (simplified)

```
CPlayer
 ├─ sock, type, key, iterator, id, accountname, nickname, version, levelname
 ├─ level* (CLevel*)
 ├─ sendbuff / outgoing
 └─ stats, weapons, openChests, enteredLevels, ...

CLevel
 ├─ filename, tiles[64*64], links, baddies, npcs, chests, signs
 ├─ LoadedMaps (static map)
 └─ load_graal() / animate()

CNpc / CBaddy / CWeapon (stubs or partial)
```

4. Data stores

- `data/` tree walked once at boot (`getSubDirs`). `getDataFile(name)` returns first match by basename.
- `accounts/<name>` — plain text, load is stubbed to spawn on `level13.graal`.
- `Weapons.txt` — INI-style catalog loaded into `weaponList`.

Default listen port: **14822**. Start map: **level13.graal**.

See ALGORITHMS.md for wire format, scramble, GCHAR, and GR-V1 details. See OPCODE-MAP.md (already present) for the packet tables.

`ash/ARCHIVAL-NOTE.md`

Archival Note — 39ster 2.1 GServer Analysis (October 2026)

This directory contains reverse-engineered architecture notes from the decommissioned Graal 39sters gServer analysis project.

What is preserved here

- `ARCHITECTURE.md` — system context, process model, class graph, data stores.
- `INDEX.md` — pack overview.

- `ALGORITHMS.md` — GCHAR codec, zlib framing, PLAYER descramble (LCG `iterator = iterator * 0x8088405 + key`, start `0x04A80B38`), GR-V1 tile bitstream, ID generation, outbound/inbound packet handling.
- Existing `OPCODE-MAP.md` already cross-references 39ster names against modern tables.

Prior attempts documented

The original analysis produced:

1. A complete museum copy of the 39ster 2.1 source tree + debug data (`39sters-2.1-Gserver/`).
2. The Ash blueprint pack (these files).
3. A C++ reference rebuild called **Phoenix** that implemented:
    - GR-V1 `.graal` loading
    - WANTGIF file serve
    - Account txt save/load
    - Board-modify and combat packet relays
    - RC account editor opcodes 70–87
    - Protocol: 2-byte BE length + zlib + newline-separated messages
    - Player decrypt via the LCG above

Phoenix had no `.nw` parser, no GS script VM, and only a listserver stub.

Full archive

The complete original data, Phoenix source, and earlier analysis packs were zipped as **Graal 39sters gServer.zip** and placed on Google Drive under `/[Grok/[Projects/` on 2026-10-28.

This PR stores the key archival notes so they remain discoverable inside Graal_analysis without requiring the external zip.

Suggested use for Seihai / future work

- Align client scramble and framing notes against the exact 2006 LCG and GCHAR rules.
- Use the GR-V1 bitstream description when validating level loaders.
- The start map for the stub account loader was `level13.graal`.
- Default port was 14822.

These notes describe the **2006 server as written**. Later open-source servers (GServer-v2, etc.) evolved beyond it.

`ash/BLUEPRINT.md`

39ster 2.1 — engineering blueprint

Source tree

```
39ster's 2.1 Gserver/
  win32Gserver.sln
  win32Gserver/          ← compile unit
    main.cpp / main.h
    sockets.cpp / sockets.h
```

```
    tools.cpp / tools.h
    ini.cpp / ini.h
    player.cpp / player.h
    level.cpp / level.h
    npc.cpp / npc.h
    includes.h
    zlib.h  zconf.h
  debug/                ← runtime cwd of the shipped exe
    win32Gserver.exe
    zlib1.dll
    Weapons.txt
    readme.txt
    developmentlog.txt
    accounts/<name>
    data/
      Levels/           .graal maps + weapon scripts as .txt
      gifs/  ganis/  charset/
      tempnpcs/
```

`getSubDirs("data/")` walks every subdirectory once at boot and stores full paths in `subdirs`. `getDataFile(name)` returns the first existing `subdir + name`. Maps are requested by basename (`level13.graal`), not by relative path.

File ownership

File	Lines (approx)	Owns
`includes.h`	6	WIN32 vs libc
`main.cpp`	259	port 14822, accept, 1s tick, idle 300s, `gameTime`, weapons INI, `getDataFile`
`sockets.cpp`	111	`tcplisten`/`tcpaccept`/`setsync`/ `tcpReceive`/`tcpSend`
`tools.cpp`	465	`CList` growth +5, `CBuffer` GCHAR codec, file load/save
`ini.cpp`	95	`[section]` / `key=value` / `#` comments
`player.h`	423	89 client ops, server ops, player props, `CPlayer`
`player.cpp`	1401	`msgFuncs`, scramble, login, warp, file/weapon send, props
`level.cpp`	451	GR-V1 decoder, sections, animate, baddy props
`npc.cpp`	176	NPC ctor + prop getters; `setProperties` almost empty

Wire frame blueprint

```
TCP stream
┌─────────────┬──────────────────────────────┐
│ len u16 BE │ zlib payload  (len bytes)    │  repeated
└─────────────┴──────────────────────────────┘

inflated payload = newline-separated messages
```

Login line (first message, **not** scrambled):

```
type:u8_gchar
[key:u8_gchar]               only if type == CLIENTPLAYER
version: 7 bytes (player) or 8 bytes (RC)   raw
mystery: 1 raw byte          skipped
namelen:u8_gchar  name:raw
passlen:u8_gchar  pass:raw
```

Later PLAYER line after descramble: `opcode:u8_gchar  payload…`

Server → client special ops (not in the named enum)

Byte	Role	Payload
100	announced length	GCHAR-int32 of the following packet size
101	raw board	4096 native `short` tiles
102	file blob	GCHAR-int64 mtime, GCHAR-len, name, raw bytes

Full runtime layout, login/presence blueprint, known holes, and remaining details are in the Drive archive under `Ash/BLUEPRINT.md`.

`ash/INDEX.md`

Ash — 39ster 2.1 GServer architecture pack

Source of truth (do not edit the museum):

`39sters-2.1-Gserver/39ster's 2.1 Gserver/win32Gserver/`

This folder is a reverse-engineered blueprint of that 2006 tree: modules, wire format, packet tables, level codec, and the algorithms the server actually runs. Comments in the original headers lie in a few places (opcode numbers after 83). The `msgFuncs[89]` table and the C enums (which increment by one) are authoritative.

File	What it is
ARCHITECTURE.md	System view, process model, class graph, data stores
BLUEPRINT.md	File-by-file ownership, runtime layout, wire frames, known holes
FLOWCHARTS.md	Mermaid flowcharts: boot, tick, login, warp, animate, files
ALGORITHMS.md	Pseudocode for scramble, zlib frames, GCHAR, GR-V1 tiles, IDs
OPCODE-MAP.md	Client / server / prop / baddy / NPC tables (already present in repo)
diagrams.html	Same mermaid diagrams in a browser-renderable page

Phoenix rebuild notes lived separately. Ash describes the **2006 server as written**, not the rewrite.

Default listen port: **14822**. Start map used by the stub account loader: **level13.graal**.

This archival copy was added from the decommissioned Graal 39sters gServer analysis project (October 2026).

`ash/OPCODE-MAP.md`

Opcode map

Date: 2026-10-21

Full session-opcode table that `protocol/OPCODE-SUMMARY.md` points to. One row per packet id, using the gs2lib names from `protocol/packet-ids.md` (same ids and 2.05 coverage). The 39ster 2.1 column lists the older names that OPCODE-SUMMARY uses, where they differ. This file covers session packets only; GS1 script opcodes are in `npc-library/gs1-reference-205.md`.

Encoding: wire byte = id + 0x20. Each message is GCHAR id + payload + `\n`. Field types: `c` = GCHAR, `s` = GSHORT, `i3`/`i4`/`i5` = 3/4/5-byte G-ints, `str` = GCHAR length + bytes, "rest" = the remainder of the line. ×2 means half-tile units. See `protocol/codec-and-framing.md`.

2.05 column: **yes** = handled (S→C) or sent (C→S) by client 2.05 rev 2; **via 100** = only reachable behind `PLO_RAWDATA`; **—** = not present in 2.05. Payloads are from GServer-v2 / gs2lib source [srv] unless marked [inf]. Empty payload cells mean the layout is not documented in these notes yet.

Client → server (PLI)

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
0	0x20	`PLI_LEVELWARP`		C→S	`c x×2, c y×2` + rest = level (byte 0 of the first line is the client type)	yes
1	0x21	`PLI_BOARDMODIFY`		C→S	`c x, c y, c w, c h` + w×h `s tile`	yes
2	0x22	`PLI_PLAYERPROPS`		C→S	PLPROP stream	yes
3	0x23	`PLI_NPCPROPS`		C→S	`i3 id` + NPCPROP stream	yes
4	0x24	`PLI_BOMBADD`	`ADDBOMB`	C→S	`c x×2, c y×2, c power, c fuse`	yes
5	0x25	`PLI_BOMBDEL`		C→S	as 4	yes
6	0x26	`PLI_TOALL`		C→S	`str message`; 2.05 sends it for chat-bar text starting with `toall:` (the rest of the text)	yes
7	0x27	`PLI_HORSEADD`		C→S	`c x×2, c y×2, c dir+4×bushes` + rest = image	yes
8	0x28	`PLI_HORSEDEL`		C→S	`c x×2, c y×2`	yes
9	0x29	`PLI_ARROWADD`		C→S	arrow: x, y, dir	yes
10	0x2a	`PLI_FIRESPY`		C→S	fire spy	yes
11	0x2b	`PLI_THROWCARRIED`		C→S	throw carried	yes
12	0x2c	`PLI_ITEMADD`		C→S	`c x×2, c y×2, c item`	yes
13	0x2d	`PLI_ITEMDEL`		C→S	`c x×2, c y×2` (no item byte; the server looks the item up by position)	yes
14	0x2e	`PLI_CLAIMPKER`		C→S	`s killer id`	yes
15	0x2f	`PLI_BADDYPROPS`		C→S	`c id` + BDPROP stream	yes
16	0x30	`PLI_BADDYHURT`		C→S	`c id` + hit data; GServer does not parse it and forwards the body unchanged to the level leader as `PLO_BADDYHURT`	yes

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
17	0x31	PLI_BADDYADD		C→S	c x×2, c y×2, c type, c power + rest = image (no id; the server assigns one, caps power at 12 and answers with PLO_BADDYPROPS)	yes
18	0x32	PLI_FLAGSET		C→S	rest = name or name=value	yes
19	0x33	PLI_FLAGDEL		C→S	rest = flag name	yes
20	0x34	PLI_OPENCHEST		C→S	c x, c y	yes
21	0x35	PLI_PUTNPC		C→S	str image, str code, c x×2, c y×2	yes
22	0x36	PLI_NPCDEL		C→S		yes
23	0x37	PLI_WANTFILE	WANTGIF	C→S	rest = filename	yes
24	0x38	PLI_SHOWIMGPLAYER		C→S	2.31 also sends it by itself about every 5.1 s, 1 byte (value 9 seen); 2.22 only with another player in the level; relayed unchanged as 32; meaning unknown **[cap]**	yes
25	0x39	PLI_UNKNOWN25		C→S		—
26	0x3a	PLI_HURTPLAYER		C→S	s victim, c dx, c dy, c power, i3 npc	yes
27	0x3b	PLI_EXPLOSION		C→S	c radius, c x×2, c y×2, c power	yes
28	0x3c	PLI_PRIVATEMESSAGE		C→S	s count, count × s id, rest = message	yes
29	0x3d	PLI_NPCWEAPONDEL		C→S	rest = weapon name	yes
30	0x3e	PLI_LEVELWARPMOD	MODLEVELWARP	C→S	i5 modtime, c x×2, c y×2 + rest = level	yes
31	0x3f	PLI_PACKETCOUNT		C→S		—
32	0x40	PLI_ITEMTAKE		C→S		yes
33	0x41	PLI_WEAPONADD		C→S	c type + (c item	i3 npc id)
34	0x42	PLI_UPDATEFILE		C→S	i5 modtime + rest = filename	yes
35	0x43	PLI_ADJACENTLEVEL		C→S	i5 modtime + rest = level	yes
36	0x44	PLI_HITOBJECTS		C→S	power, x, y [, npc]	yes
37	0x45	PLI_LANGUAGE		C→S	rest = language name	yes
38	0x46	PLI_TRIGGERACTION		C→S	i3 npc, c x×2, c y×2 + rest = action,params	yes

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
39	0x47	`PLI_TAMPERCHECK`		C→S	10 raw bytes, constant per client build; two chained CRC-32 words over the client's own image (`client-internals/decomp-findings-2026-10-24-b/01-tamper-check-id39.md`) **[cap][bin]**	—
40	0x48	`PLI_SHOOT`		C→S		—
41	0x49	`PLI_SERVERWARP`		C→S	server name; answered with `PLO_SERVERWARP` (178) **[cap]**	—
43	0x4b	`PLI_MUTEPLAYER`		C→S	`s player id`, `c 1/0` **[cap]**	—
44	0x4c	`PLI_PROCESSLIST`		C→S	the client's reply to `PLO_LISTPROCESSES` (182) **[bin][cap]**	—
45	0x4d	`PLI_PROCESSRUNNING`		C→S	`c flag`: 1 if the server's string occurs in the client's own process list **[bin]**	—
46	0x4e	`PLI_ENTERLEVEL`		C→S	`c 1`; sent by 2.31 on every level entry (login, gmap crossings, doors, links, server warps), just before the first props of the new level; not sent by 2.17/2.22 **[cap]**	—
47	0x4f	`PLI_VERIFYWANTSEND`		C→S	`i5 crc32` + rest = filename	—
48	0x50	`PLI_SHOOT2`		C→S		—
50	0x52	`PLI_RAWDATA`		C→S		—
51	0x53	`PLI_RC_SERVEROPTIONSGET`		C→S		—
52	0x54	`PLI_RC_SERVEROPTIONSSET`		C→S		—
53	0x55	`PLI_RC_FOLDERCONFIGGET`		C→S		—
54	0x56	`PLI_RC_FOLDERCONFIGSET`		C→S		—
55	0x57	`PLI_RC_RESPAWNSET`		C→S		—
56	0x58	`PLI_RC_HORSELIFESET`		C→S		—
57	0x59	`PLI_RC_APINCREMENTSET`		C→S		—
58	0x5a	`PLI_RC_BADDYRESPAWNSET`		C→S		—
59	0x5b	`PLI_RC_PLAYERPROPSGET`		C→S		—
60	0x5c	`PLI_RC_PLAYERPROPSSET`		C→S		—
61	0x5d	`PLI_RC_DISCONNECTPLAYER`		C→S		—

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
62	0x5e	PLI_RC_UPDATELEVELS		C→S		—
63	0x5f	PLI_RC_ADMINMESSAGE		C→S		—
64	0x60	PLI_RC_PRIVADMINMESSAGE		C→S		—
65	0x61	PLI_RC_LISTRCS		C→S		—
66	0x62	PLI_RC_DISCONNECTRC		C→S		—
67	0x63	PLI_RC_APPLYREASON		C→S		—
68	0x64	PLI_RC_SERVERFLAGSGET		C→S		—
69	0x65	PLI_RC_SERVERFLAGSSET		C→S		—
70	0x66	PLI_RC_ACCOUNTADD		C→S		—
71	0x67	PLI_RC_ACCOUNTDEL		C→S		—
72	0x68	PLI_RC_ACCOUNTLISTGET		C→S		—
73	0x69	PLI_RC_PLAYERPROPSGET2		C→S		—
74	0x6a	PLI_RC_PLAYERPROPSGET3		C→S		—
75	0x6b	PLI_RC_PLAYERPROPSRESET		C→S		—
76	0x6c	PLI_RC_PLAYERPROPSSET2		C→S		—
77	0x6d	PLI_RC_ACCOUNTGET		C→S		—
78	0x6e	PLI_RC_ACCOUNTSET		C→S		—
79	0x6f	PLI_RC_CHAT		C→S		—
80	0x70	PLI_PROFILEGET		C→S	rest = account	yes
81	0x71	PLI_PROFILESET		C→S	account + profile fields [inf]	yes
82	0x72	PLI_RC_WARPPLAYER		C→S		—
83	0x73	PLI_RC_PLAYERRIGHTSGET		C→S		—
84	0x74	PLI_RC_PLAYERRIGHTSSET		C→S		—
85	0x75	PLI_RC_PLAYERCOMMENTSGET		C→S		—
86	0x76	PLI_RC_PLAYERCOMMENTSSET		C→S		—
87	0x77	PLI_RC_PLAYERBANGET		C→S		—
88	0x78	PLI_RC_PLAYERBANSET		C→S		—

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
89	0x79	PLI_RC_FILEBROWSER_START		C→S		—
90	0x7a	PLI_RC_FILEBROWSER_CD		C→S		—
91	0x7b	PLI_RC_FILEBROWSER_END		C→S		—
92	0x7c	PLI_RC_FILEBROWSER_DOWN		C→S		—
93	0x7d	PLI_RC_FILEBROWSER_UP		C→S		—
94	0x7e	PLI_NPCSERVERQUERY		C→S		—
96	0x80	PLI_RC_FILEBROWSER_MOVE		C→S		—
97	0x81	PLI_RC_FILEBROWSER_DELETE		C→S		—
98	0x82	PLI_RC_FILEBROWSER_RENAME		C→S		—
103	0x87	PLI_NC_NPCGET		C→S		—
104	0x88	PLI_NC_NPCDELETE		C→S		—
105	0x89	PLI_NC_NPCRESET		C→S		—
106	0x8a	PLI_NC_NPCSCRIPTGET		C→S		—
107	0x8b	PLI_NC_NPCWARP		C→S		—
108	0x8c	PLI_NC_NPCFLAGSGET		C→S		—
109	0x8d	PLI_NC_NPCSCRIPTSET		C→S		—
110	0x8e	PLI_NC_NPCFLAGSSET		C→S		—
111	0x8f	PLI_NC_NPCADD		C→S		—
112	0x90	PLI_NC_CLASSEDIT		C→S		—
113	0x91	PLI_NC_CLASSADD		C→S		—
114	0x92	PLI_NC_LOCALNPCSGET		C→S		—
115	0x93	PLI_NC_WEAPONLISTGET		C→S		—
116	0x94	PLI_NC_WEAPONGET		C→S		—
117	0x95	PLI_NC_WEAPONADD		C→S		—
118	0x96	PLI_NC_WEAPONDELETE		C→S		—
119	0x97	PLI_NC_CLASSDELETE		C→S		—

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
130	0xa2	PLI_REQUESTUPDATEBOARD		C→S		—
150	0xb6	PLI_NC_LEVELLISTGET		C→S		—
151	0xb7	PLI_NC_LEVELLISTSET		C→S		—
152	0xb8	PLI_REQUESTTEXT		C→S		—
154	0xba	PLI_SENDTEXT		C→S		—
155	0xbb	PLI_RC_LARGEFILESTART		C→S		—
156	0xbc	PLI_RC_LARGEFILEEND		C→S		—
157	0xbd	PLI_UPDATEGANI		C→S		—
158	0xbe	PLI_UPDATESCRIPT		C→S		—
159	0xbf	PLI_UPDATEPACKAGEREQUESTFILE		C→S		—
160	0xc0	PLI_RC_FOLDERDELETE		C→S		—
161	0xc1	PLI_UPDATECLASS		C→S		—
162	0xc2	PLI_RC_UNKNOWN162		C→S		—
252	—	PLI_SET_ENC_KEY		C→S		—
253	—	PLI_BUNDLE		C→S		—

Server → client (PLO)

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
0	0x20	PLO_LEVELBOARD		S→C	empty = use cached board; else board changes (as 7)	yes
1	0x21	PLO_LEVELLINK		S→C	rest = `dest x y w h newx newy`	yes
2	0x22	PLO_BADDYPROPS	SBADDYPROPS	S→C	`c id` + BDPROP stream	yes
3	0x23	PLO_NPCPROPS	SNPCPROPS	S→C	`i3 id` + NPCPROP stream	yes
4	0x24	PLO_LEVELCHEST		S→C	`c opened, c x, c y` [+ `c item, c signindex`]	yes
5	0x25	PLO_LEVELSIGN		S→C	`c x, c y` + sign-alphabet text	yes
6	0x26	PLO_LEVELNAME		S→C	rest = level filename	yes
7	0x27	PLO_BOARDMODIFY	SBOARDMODIFY	S→C	`c x, c y, c w, c h` + w×h `s tile`	yes
8	0x28	PLO_OTHERPLPROPS		S→C	`s id` + PLPROP stream	yes
9	0x29	PLO_PLAYERPROPS	SPLAYERPROPS	S→C	PLPROP stream (self)	yes
10	0x2a	PLO_ISLEADER		S→C	empty: you lead this level	yes

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
11	0x2b	`PLO_BOMBADD`		S→C	`c x×2, c y×2, c power, c fuse`	yes
12	0x2c	`PLO_BOMBDEL`		S→C	as 11	yes
13	0x2d	`PLO_TOALL`		S→C	`s player id`, `str message`	yes
14	0x2e	`PLO_PLAYERWARP`		S→C	`c x×2, c y×2` + rest = level	yes
15	0x2f	`PLO_WARPFAILED`		S→C	rest = level	yes
16	0x30	`PLO_DISCMESSAGE`		S→C	rest = text; connection closes	yes
17	0x31	`PLO_HORSEADD`		S→C	relayed PLI 7 body: `c x×2, c y×2, c dir+4×bushes` + image	yes
18	0x32	`PLO_HORSEDEL`		S→C	`c x×2, c y×2`	yes
19	0x33	`PLO_ARROWADD`		S→C	arrow: x, y, dir	yes
20	0x34	`PLO_FIRESPY`		S→C	fire spy (relayed)	yes
21	0x35	`PLO_THROWCARRIED`		S→C	throw carried (relayed)	yes
22	0x36	`PLO_ITEMADD`		S→C	`c x×2, c y×2, c item`	yes
23	0x37	`PLO_ITEMDEL`		S→C	`c x×2, c y×2, c item`	yes
24	0x38	`PLO_NPCMOVED`		S→C		yes
25	0x39	`PLO_SIGNATURE`		S→C	`c v`: 1 = the client quits; 0 = quits unless the host is the loopback address; other values ignored (layout read in 2.17–2.31; the id is handled from 2.00) **[bin]**	yes
26	0x3a	`PLO_NPCACTION`		S→C		—
27	0x3b	`PLO_BADDYHURT`		S→C	baddy hurt: `c id` + BDPROP stream	yes
28	0x3c	`PLO_FLAGSET`		S→C	rest = `name` or `name=value`	yes
29	0x3d	`PLO_NPCDEL`		S→C		yes
30	0x3e	`PLO_FILESENDFAILED`		S→C	rest = filename (pre-2.1: also "up to date")	yes
31	0x3f	`PLO_FLAGDEL`		S→C	rest = flag name	yes
32	0x40	`PLO_SHOWIMGPLAYER`		S→C	`s sender id` + the sender's 24 payload, relayed unchanged **[cap]**	yes
33	0x41	`PLO_NPCWEAPONADD`		S→C	`str name`, NPCPROP image + script	yes
34	0x42	`PLO_NPCWEAPONDEL`		S→C	rest = weapon name	yes
35	0x43	`PLO_RC_ADMINMESSAGE`		S→C	rest = `Admin <account>: \xa7<text>`	yes
36	0x44	`PLO_EXPLOSION`		S→C	`c radius, c x×2, c y×2, c power`	yes
37	0x45	`PLO_PRIVATEMESSAGE`		S→C	`s from id`, rest = quoted `"type","message"`	yes
38	0x46	`PLO_PUSHAWAY`		S→C	`c dx, c dy`	yes
39	0x47	`PLO_LEVELMODTIME`		S→C	`i5 modtime`; 2.17–2.31: the rest is passed unparsed to the active gmap or level object, only if one exists **[bin]**	yes

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
40	0x48	`PLO_HURTPLAYER`		S→C	`s attacker, c dx, c dy, c power, i3 npc`	yes
41	0x49	`PLO_STARTMESSAGE`		S→C	rest = RTF or HTML text; cached as `servermessage.rtf` / `.html` and shown in the start-message viewer; client 6.037 ignores it **[bin]**	yes
42	0x4a	`PLO_NEWWORLDTIME`		S→C	`i4 time`	yes
43	0x4b	`PLO_DEFAULTWEAPON`		S→C	`c item index`	yes
44	0x4c	`PLO_HASNPCSERVER`		S→C	no payload; handled by 2.00–2.22, not handled by 2.31 **[bin]**	yes
45	0x4d	`PLO_FILEUPTODATE`		S→C	rest = filename	yes
46	0x4e	`PLO_HITOBJECTS`		S→C	hit objects: power, x, y [, npc]	yes
47	0x4f	`PLO_STAFFGUILDS`		S→C	`"guild"`, repeated; comma text, stored as the staff-guild list **[bin]**	yes
48	0x50	`PLO_TRIGGERACTION`		S→C	`i3 npc, c x×2, c y×2` + rest = `action,params`	yes
49	0x51	`PLO_PLAYERWARP2`		S→C	`c x×2, c y×2, c z` (unused by 2.17–2.31), `c gmapX, c gmapY`, rest = level; x and y are `v/2` tiles; gmapX/Y apply only when the level name ends in `.gmap` (clamped to the grid) **[bin]**	—
50	0x52	`PLO_RC_ACCOUNTADD`		S→C		—
51	0x53	`PLO_RC_ACCOUNTSTATUS`		S→C		—
52	0x54	`PLO_RC_ACCOUNTNAME`		S→C		—
53	0x55	`PLO_RC_ACCOUNTDEL`		S→C		—
54	0x56	`PLO_RC_ACCOUNTPROPS`		S→C		—
55	0x57	`PLO_ADDPLAYER`		S→C		—
56	0x58	`PLO_DELPLAYER`		S→C		—
57	0x59	`PLO_RC_ACCOUNTPROPSGET`		S→C		—
58	0x5a	`PLO_RC_ACCOUNTCHANGE`		S→C		—
59	0x5b	`PLO_RC_PLAYERPROPSCHANGE`		S→C		—
60	0x5c	`PLO_UNKNOWN60`		S→C		—
61	0x5d	`PLO_RC_SERVERFLAGSGET`		S→C		—

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
62	0x5e	`PLO_RC_PLAYERRIGHTSGET`		S→C		—
63	0x5f	`PLO_RC_PLAYERCOMMENTSGET`		S→C		—
64	0x60	`PLO_RC_PLAYERBANGET`		S→C		—
65	0x61	`PLO_RC_FILEBROWSER_DIRLIST`		S→C		—
66	0x62	`PLO_RC_FILEBROWSER_DIR`		S→C		—
67	0x63	`PLO_RC_FILEBROWSER_MESSAGE`		S→C		—
68	0x64	`PLO_LARGEFILESTART`		S→C	rest = file name (2.31 also clears the stored size) **[bin]**	—
69	0x65	`PLO_LARGEFILEEND`		S→C	rest = file name; if it equals the stored name the received stream is processed as a file **[bin]**	—
70	0x66	`PLO_RC_ACCOUNTLISTGET`		S→C		—
71	0x67	`PLO_RC_PLAYERPROPS`		S→C		—
72	0x68	`PLO_RC_PLAYERPROPSGET`		S→C		—
73	0x69	`PLO_RC_ACCOUNTGET`		S→C		—
74	0x6a	`PLO_RC_CHAT`		S→C	RC console line	—
75	0x6b	`PLO_PROFILE`		S→C	repeat `c n` + n chars; first field = account, the rest = profile lines **[bin]**	yes
76	0x6c	`PLO_RC_SERVEROPTIONSGET`		S→C		—
77	0x6d	`PLO_RC_FOLDERCONFIGGET`		S→C		—
78	0x6e	`PLO_NC_CONTROL`		S→C		—
79	0x6f	`PLO_NPCSERVERADDR`		S→C		—
80	0x70	`PLO_NC_LEVELLIST`		S→C		—
81	0x71	`PLO_UNKNOWN81`		S→C		—
82	0x72	`PLO_SERVERTEXT`		S→C		—
83	0x73	`PLO_UNKNOWN83`		S→C		—
84	0x74	`PLO_LARGEFILESIZE`		S→C	`i5 size` (2.31 only) **[bin]**	—
100	0x84	`PLO_RAWDATA`	`FILELEN`	S→C	`i3 length` of the next raw message	yes
101	0x85	`PLO_BOARDPACKET`	`TILES`	S→C	4096 × u16 LE tiles (behind 100)	via 100

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
102	0x86	`PLO_FILE`	`FILEBLOB`	S→C	file: `str name` [+ `i5 modtime` ≥ 2.1] + bytes (behind 100)	via 100
103	0x87	`PLO_RC_MAXUPLOADFILESIZE`		S→C		—
104	0x88	`PLO_UNKNOWN104`		S→C		—
105	0x89	`PLO_UPDATEPACKAGESIZE`		S→C		—
106	0x8a	`PLO_UPDATEPACKAGEDONE`		S→C		—
107	0x8b	`PLO_BOARDLAYER`		S→C		—
108	0x8c	`PLO_UNKNOWN108`		S→C		—
109	0x8d	`PLO_UNKNOWN109`		S→C		—
110	0x8e	`PLO_UNKNOWN110`		S→C		—
111	0x8f	`PLO_SETNETCOOKIE`		S→C		—
124	0x9c	`PLO_UNKNOWN124`		S→C		—
131	0xa3	`PLO_NPCBYTECODE`		S→C		—
132	0xa4	`PLO_UNKNOWN132`		S→C		—
133	0xa5	`PLO_UNKNOWN133`		S→C		—
134	0xa6	`PLO_GANISCRIPT`		S→C		—
140	0xac	`PLO_NPCWEAPONSCRIPT`		S→C		—
150	0xb6	`PLO_NPCDEL2`		S→C	`c n`, n-char level name, `i3 npc id` **[bin]**	yes
151	0xb7	`PLO_HIDENPCS`		S→C	no payload; removes NPCs with ids 1 to 999 of the active level (id 0 excluded) **[bin]**	yes
153	0xb9	`PLO_SAY2`		S→C	rest = text, `\xa7` = newline	yes
154	0xba	`PLO_FREEZEPLAYER2`		S→C	empty	yes
155	0xbb	`PLO_UNFREEZEPLAYER`		S→C	empty	yes
156	0xbc	`PLO_SETACTIVELEVEL`		S→C	rest = level or `.gmap` name; a `.gmap` is loaded and made active. 2.31 adds an extra follow-up step (purpose unknown) **[bin]**	yes
157	0xbd	`PLO_NC_NPCATTRIBUTES`		S→C		—
158	0xbe	`PLO_NC_NPCADD`		S→C		—
159	0xbf	`PLO_NC_NPCDELETE`		S→C		—

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
160	0xc0	`PLO_NC_NPCSCRIPT`		S→C		—
161	0xc1	`PLO_NC_NPCFLAGS`		S→C		—
162	0xc2	`PLO_NC_CLASSGET`		S→C		—
163	0xc3	`PLO_NC_CLASSADD`		S→C		—
164	0xc4	`PLO_NC_LEVELDUMP`		S→C		—
165	0xc5	`PLO_MOVE`		S→C	`i3 npc id` (> 0) + move data **[bin]**	yes
166	0xc6	`PLO_SHOWIMGNPC`		S→C	`i3 npc id` + prop list **[bin]**	yes
167	0xc7	`PLO_NC_WEAPONLISTGET`		S→C		—
168	0xc8	`PLO_UNKNOWN168`		S→C	payload ignored; local-player reset (2.17–2.31) **[bin]**	yes
169	0xc9	`PLO_UPGRADEURL`		S→C	rest = URL text **[bin]**	—
170	0xca	`PLO_GHOSTMODE`		S→C	`c flag` (> 0 = ghost on) **[bin]**	—
171	0xcb	`PLO_BIGMAP`		S→C	comma text `txt,png,x,y` (≥ 4 fields used); x, y parsed as floats **[bin]**	—
172	0xcc	`PLO_MINIMAP`		S→C	same layout as 171, goes to the minimap **[bin]**	—
173	0xcd	`PLO_GHOSTTEXT`		S→C	rest = text **[bin]**	—
174	0xce	`PLO_GHOSTICON`		S→C	`c flag` (> 0 = show icon) **[bin]**	—
175	0xcf	`PLO_SHOOT`		S→C	`s player id` + shot data; only when the player is in the active level **[bin]**	—
176	0xd0	`PLO_KINGDOMSMODE`		S→C	no payload **[bin]**	—
177	0xd1	`PLO_NEWWORLDMODE`		S→C	no payload **[bin]**	—
178	0xd2	`PLO_SERVERWARP`		S→C	comma text, ≥ 4 fields: 0 unused, 1 name, 2 host, 3 port; client reconnects with a fresh login (2.31 lab) **[bin][cap]**	—
179	0xd3	`PLO_RPGWINDOW`		S→C	rest = text **[bin]**	—
180	0xd4	`PLO_STATUSLIST`		S→C	rest = text **[bin]**	—
181	0xd5	`PLO_UNKNOWN181`		S→C	`s player id`, `c toggle` (length ≥ 3) **[bin]**	—
182	0xd6	`PLO_LISTPROCESSES`		S→C	no payload; client answers `PLI_PROCESSLIST` (44) **[bin]**	—
183	0xd7	`PLO_HASPROCESSRUNNING`		S→C	rest = names; client answers `PLI_PROCESSRUNNING` (45) with `c 0/1` **[bin]**	—

Id	Wire	Name	39ster 2.1	Dir	Payload	2.05
184	0xd8	`PLO_TAKESCREENSHOT`		S→C	rest = file name, only `.png` accepted; saved under a screenshots folder **[bin]**	—
185	0xd9	`PLO_BOARDHEIGHTS`		S→C	`c mapX, c mapY, c blockX, c blockY, c w-1, c h-1`, then w×h cells of 2 raw bytes; height = (b0−0x20−50) + (b1−0x20)/128 **[bin]**	—
186	0xda	`PLO_BOARDMODIFY2`		S→C	`c mapX, c mapY, c x, c y, c w, c h`, then w×h `s tile` row-major, then ignored padding: bytes after the tiles are not read, a payload one byte short of 6+2*wh* is dropped silently, a block past column 63 is clipped (2.22+; client reaction lab-tested with injected frames) **[bin][cap]**	—
187	0xdb	`PLO_UPDATEPACKAGEISUPDATED`		S→C		—
188	0xdc	`PLO_NC_CLASSDELETE`		S→C		—
189	0xdd	`PLO_MOVE2`		S→C	as 165 with an extra flag (2.31 only) **[bin]**	—
190	0xde	`PLO_SERVERLISTCONNECTED`		S→C		—
191	0xdf	`PLO_SHOOT2`		S→C		—
192	0xe0	`PLO_NC_WEAPONGET`		S→C		—
193	0xe1	`PLO_UNKNOWN193`		S→C		—
194	0xe2	`PLO_CLEARWEAPONS`		S→C		—
195	0xe3	`PLO_LOADGANI`		S→C		—
197	0xe5	`PLO_LOADSCRIPT`		S→C		—
198	0xe6	`PLO_SERVEROPTIONS`		S→C		—
252	—	`PLO_SET_ENC_KEY`		S→C		—
253	—	`PLO_BUNDLE`		S→C		—

Counts: 128 PLI ids, 38 sent by 2.05; 152 PLO ids, 59 handled by 2.05 (plus 101/102 behind 100).

Notes: - 39ster 2.1 numbered RC ops 51–87 as mostly empty stubs. In gs2lib these are `PLI_RC_*` (51–98) and `PLI_NC_*` (103–119). 2.05 sends none of them. - The 2.05 peer-to-peer UDP channel has its own id space; see `protocol/udp-p2p.md`. - The first client line is not PLI 0: byte 0 is the client type (0 = classic client). See `protocol/login-and-walk.md`.

CONSOLIDATED.md

Graal Analysis — consolidated notes

Date: 2026-10-14, revised 2026-10-22

Cleaned notes derived from public server sources (GServer-v2, gs2lib, graal-serverlist, OpenGS1), observed packet shapes, format documentation, and behavior observed in client 2.05 rev 2. No decompile material.

Formats

Levels

- **GLEVNW01** (`.nw`) — text format. Header, `BOARD x y width layer tiledata` (2-char base64 tiles), `NPC` blocks, `LINK`/`SIGN`/`CHEST`/`BADDY` records.
- **GR-V1** (`.graal`) — binary format. 8-byte magic, then a 12-bit (1.00, 1.01) or 13-bit (1.02, 1.03) LSB-first tile bit stream with RLE and double-RLE control codes, then text sections for links, baddies, NPCs (`image#script`), chests, and signs. See `formats/level-binary.md`.
- **Z3-V1** (`.zelda`) — earlier binary. RLE board (11-bit for 1.02, 12-bit for 1.03/1.04). FileLink records until `#`, then binary computer records and coded signs. No NPCs or chests. Seihai loads the board; trailing records are not yet parsed.

Tiles

- Tile id is a 12-bit value decoded from the 2-char base64 encoding into `pics1`.
- Collision table is 4096 bytes. Wall is `>= 20` (blocks). Water is `== 11` (does not block, changes display).
- `arrays.dat` first array is the passable-tile list (`nowall`). Tiles in that list get type 0; everything else gets type 22.
- Stamp id 0 is a 4×4 ground fill. Stamp id 1 is a hole and is not written. The 2×2 Bush stamp writes ids 2, 3, 18, 19.

GANI

- `SPRITE`/`ANI`/`ANIEND`/`PLAYSOUND`/`DEFAULTATTR` blocks.
- `ANI` carries placements. A stepper advances the frame index.

Maps

- **GRMAP001** (`.gmap`) — world map linking levels. Reader keys are in `formats/gmap.md`. 2.05 rev 2 has the `GRMAP001` string and a reader, but no static caller was found; its active path is `bigmap.txt`/`minimap.txt`.

Protocol

Framing

- Frame = u16be length + zlib body (zlib 1.1.2 in 2.05). Message = GCHAR id + payload + `\n`. Binary payloads go behind `PLO_RAWDATA` (100).
- **2.05 = gs2lib gen 3:** the client inserts one byte per message from the second message on. The iterator is seeded `0x04A80B38` and stepped `it = it*0x08088405 + key`. The byte `((it>>16)&0x7F)+0x20` goes

in at 1-based position `(it&0xFFFF) % len + 1`, which can be 1 (before the id byte). Only the login line is clean. Server→client is plain zlib. The insertion is applied per message before the buffer is compressed.
- Full-frame partial XOR with the same LCG is gen 4/5 (2.19+ clients), not 2.05. See `protocol/codec-and-framing.md`.

Login

- First line: GCHAR client type (0) + GCHAR key (`rand & 0x7F`, the old "random prefix") + `GNW#####` + GSTRING account + GSTRING password. Then `PLI_LANGUAGE` (37).
- Observed tokens: `GNW28015`, `GNW22122`, `GNW03014`, `GNW07080`, `GNW30123`.
- `GNW07080` = 1.39. `GNW03081` = 2.05 rev 2 (CLVER_2_052, pre-2.1 paths). `GNW22122` = 2.17, `GNW03014` = 2.22, `GNW28015` = 2.31. `GNW30123` is the 2.22/2.31 listserver token (see `protocol/client-versions.md`).

Movement

- The short frames headed wire `"` are `PLI_PLAYERPROPS` (2) prop streams: 15 X, 16 Y, 17 SPRITE (GCHAR, half-tiles). Not f32.
- Position codecs: classic ×2 (props 15/16), precise ×16 (props 78/79, newer clients).
- Directions: 0 up, 1 left, 2 down, 3 right (corrected).

Board

- `PLO_BOARDMODIFY` (7) / `PLI_BOARDMODIFY` (1): GCHAR x, y, w, h + w×h GSHORT tiles. `PLO_BOARDPACKET` (101, behind RAWDATA) = 4096 raw LE u16.
- Packet ids are settled: the 2.05 receive jump table matches gs2lib `PLO_*` 0–25, 27–48, 75, 100, 150, 151, 153–156, 165, 166, 168. Client senders cover 0–38 (minus 6, 25, 31), 80, 81. See `protocol/packet-ids.md`.

NPC scripting

3,172 unique scripts scraped from 1,420 GR-V1 levels. Stored in `npc-library/npcs.json`. The scrape reported 3,187; 15 records were lost to truncated source pages (see `partial_pages` in `npcs.json` meta).

Common events (by frequency)

Event	Scripts
`playerenters`	2,268
`playertouchsme`	1,986
`timeout`	1,456
`playerendsreading`	1,123
`compusdied`	1,122
`playertouchsother`	1,074
`wasshot`	728
`playersays`	43

Script shape

Scripts follow `image.gif#if (event) { body }`. Common commands: `hide`, `show`, `set`, `unset`, `say`, `setlevel`, `warp`, `hurt`. The 2.05 tables (161 commands, op = 600 + index, 600–760; 177 functions/vars) are in `npc-library/gs1-reference-205.md`.

GS1 bytecode

The 2.05 client does **not** use bytecode. It parses script text into a tree and interprets that recursively. The OpenGS1 opcodes below belong to Seihai's own VM, a reimplementation.

Opcodes match OpenGS1: Push, Assign, ArrSet/Get, Add/Sub/Mul/Div/Mod/Pow, Inc/Dec, Call/CmdCall, Jmp/Jal/Ret, Eq/Lt/Gt/Lte/Gte, Not/And/Or, Jez/Jnz, Stop, DbgOut. Arithmetic, comparison, and jump execute. Call and array ops are no-ops until the command set is wired.

Offline play

A local session can: 1. Load a GLEVNW01 `.nw`. 2. Decode tile ids and apply board modifies. 3. Reject moves onto wall tiles. Track water without blocking. 4. Parse LINK/SIGN/BADDY/CHEST. Decode binary sign text. 5. Step an optional walk GANI on every successful move. 6. Draw an ASCII or PPM view of the board and player.

Version map

Token	Build
`GNW07080`	1.39
`GNW03081`	2.05
`GNW22122`	2.17
`GNW28015`	2.31
`GNW03014`	2.22
`GNW30123`	2.22/2.31 listserver token

Filename builds: `Graal_2171` = 2.171, `Graal_222` = 2.22, `Graal_231` = 2.31. PE FileVersion is authoritative for editors.

Precursor

Zelda Online (1997–98, Stefan Knorr) is the Java precursor. AWT applet client/server on port 14801, text-line lobby protocol, `.zelda` levels. Source is archived separately. Levels were analyzed and discarded.

Reference notes added 2026-10-21

- `protocol/codec-and-framing.md`: G-types, frames, raw-data escape, encryption gens 1–6.
- `protocol/packet-ids.md`: full PLI/PLO tables with 2.05 coverage.
- `protocol/packet-layouts.md`: level, player, combat, navigation, and weapon payloads.
- `protocol/player-props.md`: PLPROP, NPCPROP, and BDPROP ids and encodings.
- `protocol/file-transfer.md`, `protocol/chat-pm-rc.md`, `protocol/listserver.md`, `protocol/udp-p2p.md` (2.05 peer UDP).
- `formats/level-binary.md`, `formats/gmap.md`.
- `npc-library/gs1-reference-205.md`, `client-internals/client-architecture-205.md`.

`server-notes/01-lineage.md`

Lineage of the dumps

What is actually source

Tree	Era	Shape	Lines (approx)
`linuxgserver131`	1.31	`gserver.cpp` monolith + GC headers	~9k with zlib headers
`sidrax-v1.4-srv-src-1.3.9`	1.3.9	`gserver.cpp` + `levels.cpp` + `npcs.cpp` + MySQL	~13k
`oldwin32-gserver1.39source`	1.39	same family, Win32	similar
`gservernwsrc`	NW / 1.4-ish	objects + pascal strings	medium
`TheDgtl-gs2emu-gserver-d3ecab4`	Reborn 2.x	T* classes, premake, miniupnpc/ zlib/bzip2 vendored	~18k server/src

What is binary / world only

Tree	Notes
`gserver_2.1.0`, `2.2.0-fix`, `2.3.1`, `2.4.0`	Reborn binaries + default worlds
`GServerRev29/31/51`	2.17-line packaged servers
`gserver-134-win32`, `unlgserv119`	old exe + accounts
`JustBurn`, `Matriark`, `projectx`	worlds or incomplete trees
`/Phoenix/`	Reborn **2.4.0** `gserver2.exe`, world `servers/phoenix`, port **14802**
`/Ash/`	Python rewrite of the gs2emu protocol, world `servers/ash`, port **14812**

What changed between eras

1. **1.3.x monolith**
   - One listen socket, one world.
   - Login + movement + chat live in `gserver.cpp`.
   - Levels and NPCs split out later (`levels.cpp`, `npcs.cpp`).
   - Strings are Pascal (`length byte + bytes`) via `pascalstrings.cpp`.
   - Accounts often MySQL (`graal_mysql.cpp` in sidrax) or flat files.
   - Encryption is gen 3 at most (dummy-byte + zlib).
2. **gs2emu / Reborn 2.x**
   - `servers.txt` runs N worlds in one process.
   - Player is a typed object: client, RC, NPC-Server.
   - Encryption gens 3, 4, 5 keyed off login type byte.
   - File-backed `GRACC001` accounts, `GLEVNW01` levels.
   - RC file browser and option editors.
   - Listserver protocol (`SVO_*` / `SVI_*`).
   - Optional NPC-Server process.
   - Trigger-action “hacks” (`gr.addweapon`, …).
3. **Phoenix**
   - No source. Same engine as `gserver_2.4.0`.
   - Config-only package: staff, allowed versions 2.1.7–2.3.1, offline-friendly options.
4. **Ash**
   - New code. Same on-disk layout and wire protocol.

- `protocol.py` = CString + CEncryption + CFileQueue.
- `ashserver.py` = TServer + TPlayer + TAccount + TLevel.
- `extras.py` = TServerList + TPlayerRC + TMap + TLevelBaddy + triggers + NC.

(Extracted from the Ember atlas in the decommissioned local workspace.)

`server-notes/03-protocol.md`

Protocol

All multi-byte integers on the player socket are **big-endian**. Listserver packets have **no** length prefix; they are `id + payload + \n`.

G-integers (CString writeG*)

Every “Graal packed” integer is 7-bit digits stored as `digit + 32` so the bytes land in the printable range.

```
gchar  n        ->  1 byte   (n + 32)                        max 223
gshort n        ->  2 bytes  ((n>>7)+32), (n&127)+32         max 12255
gint3  n        ->  3 bytes  14/7/0 shifts                   max 1_568_735
gint4  n        ->  4 bytes
gint5  n        ->  5 bytes  used for file size and modtime
```

`operator>> (char)` on CString writes a **gchar**, not a raw byte. `operator<< (char)` writes a raw byte. That distinction is how `PLO_SIGNATURE >> (char)73` becomes the byte `105`.

Pascal string on the wire: `gchar length` + raw bytes (not +32 on the text).

Login blob (first line, after zlib)

```
gchar typeShift
gchar key                  # clients, and RC when gen > 3
char[8] version            # "GNW28015" etc
u8 nameLen + name
u8 passLen + pass
[NPC-Server only] gshort listenPort
\n
```

Version table lives in `IUtil.cpp` / `versions.txt` / `Ash/protocol.py VERSIONS`.

Player props

A props packet is:

```
gchar PLO_PLAYERPROPS | PLO_OTHERPLPROPS
gshort playerId            # OTHERPLPROPS only
repeated:
    gchar propId
    <value whose width depends on propId>
```

Important encodings (TPlayerProps.cpp):

Id	Name	Value
0	NICKNAME	gchar len + bytes
1	MAXPOWER	gchar hearts
2	CURPOWER	gchar (hearts * 2)
8	SWORDPOWER	gchar (power+30) + pstring image
9	SHIELDPOWER	gchar (power+10) + pstring image
11	HEADGIF	gchar (len+100) + bytes
14	ID	gshort
15/16	X/Y	gchar (tile * 2)
20	CURLEVEL	pstring
34	ACCOUNTNAME	pstring
36	RATING	gint3 `((rating&0xFFF)<<9) | (deviation&0x1FF)`
78/79	X2/Y2	higher-res 2.19+

Login uses two bitmasks `__sendLogin` (self) and `__getLogin` (others).

Level board

`GLEVNW01` text levels. `BOARD x y width layer TILECHARS` — two characters per tile. Runtime board is `uint16 tiles[64*64]` sent as:

```
gchar PLO_RAWDATA
gint3  (1 + 8192 + 1)
\n
gchar PLO_BOARDPACKET
8192 raw tile bytes
\n
```

Links: `PLO_LEVELLINK` + `dest x y width height destx desty` text. Baddies: `PLO_BADDYPROPS` + gchar id + prop stream.

File send (2.1+)

```
PLO_RAWDATA  gint3 (1+5+1+namelen+1 + filesize)
PLO_FILE     gint5 modtime, gchar namelen, name, bytes, \n
```

Listserver ids

Outbound `SVO_`: SETNAME 0 … NEWSERVER 22, SERVERHQPASS 23, SERVERHQLEVEL 24, VERIACC2 17, PLYRADD 14, PLYRREM 15, SETPLYR 7, PING 16.

Inbound `SVI_`: VERIACC2 11, ERRMSG 8, PING 99, RAWDATA 100, FILESTART3/DATA3/END3 15–17 (global head/body downloads).

(Extracted from the Ember atlas in the decommissioned local workspace.)

`server-notes/04-algorithms.md`

Algorithms and pseudocode

Faithful to `CEncryption.cpp`, `TPlayer.cpp`, `TPlayerLogin.cpp`, `TServerList.cpp`, `TMap.cpp`, `TLevelBaddy.cpp`, `msgPLI_TRIGGERACTION`. Identifiers match the dumps.

1. LCG step

```
function step(codec):
    codec.iterator ← (codec.iterator × 0x8088405 + codec.key) mod 2³²

ITERATOR_START = [0, 0, 0x04A80B38, 0x04A80B38, 0x04A80B38, 0]

function reset(codec, key):
    codec.key ← key
    codec.iterator ← ITERATOR_START[codec.gen]
    codec.limit ← -1
```

2. Gen-3 dummy byte (client → server packets only)

```
function gen3_insert(codec, buf):          # client encrypt
    step(codec)
    pos ← (codec.iterator ∧ 0xFFFF) mod length(buf)
    return buf[0..pos) + byte(')') + buf[pos..]

function gen3_remove(codec, buf):          # server decrypt
    step(codec)
    pos ← (codec.iterator ∧ 0xFFFF) mod length(buf)   # length is already +1
    delete buf[pos]
```

gs2emu outbound gen 3 does **not** insert. Only zlib + length.

3. Gen-4 / gen-5 XOR stream

```
function xor_stream(codec, buf):
    i ← 0
    while i < length(buf):
        if i mod 4 = 0:
            if codec.limit = 0: return buf
            step(codec)
            if codec.limit > 0: codec.limit ← codec.limit - 1
            word ← iterator as little-endian 4 bytes
        buf[i] ← buf[i] xor word[i mod 4]
        i ← i + 1
    return buf
```

Inbound gen 4: `limitFromType(0x06)`, xor, bz2-decompress. Inbound gen 5: read type byte, `limitFromType(type)`, xor, then decompress by type.

4. Inbound frame

```
function doMain(player):
    while recvbuf has ≥ 2 bytes:
        n ← u16be(recvbuf)
        if recvbuf shorter than 2+n: break
        payload ← recvbuf[2..2+n)
        consume 2+n
        case player.in_codec.gen of
            1: inner ← payload
            2,3: inner ← zlib_dec(payload)
            default: inner ← decryptPacket(payload)
        parsePacket(player, inner)

function parsePacket(player, inner):
    if player.type = AWAIT:
        line ← read through '\n'
        return msgPLI_LOGIN(line)
    for each line in inner split '\n':
        if gen = 3 and player.isClient:
            gen3_remove(player.in_codec, line)
        id ← gchar(line[0])
        TPLFunc[id](line[1..])
```

5. Login

```
function msgPLI_LOGIN(pkt):
    type ← 1 << gchar()
    choose gen from type          # CLIENT→3, CLIENT2→4, CLIENT3/RC2→5
    if client or new RC:
        key ← gchar(); reset(in_codec, key); out_codec matches
    version ← 8 bytes
    name ← pstring(); pass ← pstring()
    if version not in allowedversions: disc("not allowed"); return false
    if ip banned: disc(); return false
    if RC and name not in staff: disc("no RC rights"); return false
    if NPCSERVER:
        if pass ≠ ns_password: disc(); return false
        setNPCServer(this, gshort port)
        sendNC_Maps(); sendNC_Levels(); sendNC_Weapons()
        return true
    send listserver SVO_VERIACC2(name, pass, id, type)
    # continue in SVI_VERIACC2
```

(See the full file in the Drive archive for the complete set of sections: warp, gmap, baddies, trigger hacks, RC file browser, gtokenize, AP tick, etc.)

(Extracted from the Ember atlas in the decommissioned local workspace.)

`server-notes/ash-overview.md`

Ash — Python reference implementation

A rewritten gserver. This is not the Reborn 2.4 binary used by Phoenix. The code in ashserver.py / protocol.py was written from a comparison of the source dumps in `graal gservers/`.

What Ash implements

- servers.txt multi-instance (one world ships: servers/ash/)
- listen socket, 2-byte length framing
- encryption gen 3 / 4 / 5
- login (client + RC + NPC-Server)
- listserver register / ping / VERIACC2 (offline_login=true still works if the list is down)
- GRACC001 accounts, GLEVNW01 levels including BADDY
- props, chat, PM, warp, files
- RC file browser, options, accounts, flags
- gmaps / bigmaps / groupmaps
- baddies (hurt / respawn / optional drops)
- triggeraction hacks
- NPC-Server HASNPCSERVER + address + query relay

Horses, weapon bytecode and UPnP are still out. Phoenix still runs the Reborn 2.4.0 binary if you need that exact engine.

Quick start

1. Rename servers/ash/accounts/YOURACCOUNT.txt to your account name.
2. Put that name on staff= in servers/ash/config/serveroptions.txt.
3. python3 ashserver.py or ./start_ash.sh
4. Point a v2 client at your IP, port 14812.

The full Ash sources and world are in the Google Drive archive linked from `../ARCHIVAL.md`.

Summary of the Ash rewrite from the decommissioned local workspace.

`server-notes/lab-servers-2026-10-30.md`

Our gserver and listserver, and the open-source servers around them (2026-10-30)

What the project's own servers are, what they do, and how they compare with GServer-v2 beta4, graal-serverlist, glauncher and gs2-parser. The open-source repositories were read, not changed: nothing was pushed, filed or proposed to any repository outside squirelworks. Their code is summarized and cited, not copied. How to run our servers is in `../tools/lab-servers/README.md`.

Tags: **[src]** read in the named source, **[lab]** run in the offline lab today with the real client named, **[inf]** inferred, not run.

1. What "our servers" are

There is no squirelworks gserver or listserver repository. The servers the project runs are the **Seihai lab pack** (`seihai-lab/servers/seihai-lab-servers.tar.gz` in the project files, built 2026-10-22):

Piece	What it is
gserver	GServer-v2 **master `efb5b90`** (2025-01-07), built with `V8NPCSERVER=OFF`, `UPNP=OFF`, against gs2lib `b839106` plus a one-line patch that puts the 2.05 token `GNW03081` in the 2.1 slot. Reports `3.0.9-beta`. Ships the stock `default` world with `onlinestartlocal.nw`. **[src]**
listserver	`listserver.py`, a Python stub (119 lines) written for the lab. Replaced today by version 2 (`../tools/lab-servers/listserver.py`).
sources kept	`seihai-lab/sources/GServer-v2.zip` (the same `efb5b90` tree) and `graal-listserver.zip`, which is **Guthius/graal-listserver** (a C# "2.1.5 list server emulator"), not xtjoeytx/graal-serverlist.

Older material (the 39ster 2.1 server, Phoenix, Ash) is in `ash/` and `server-notes/01-lineage.md`; none of it runs in the lab.

What our gserver does with each client generation [src]

- The login type byte selects the player type and cipher generation (`../protocol/client-versions.md`); v5's type 5 is `PLTYPE_CLIENT3`. A trailing identity string (`win`, hardware ids, OS) is read and forwarded to the listserver.
- Version comes from the token through gs2lib's table. An unknown token that is in `allowedversions.txt` gets `CLVER_UNKNOWN` (0), which sorts below everything, so the client is served every pre-2.1 code path. This is why 5.24 (`G3D15010`) gets the gani prop in the old form (v5 thread, 2026-10-30).
- Below `CLVER_2_1` the level goes through `sendLevel141`, which never sends `PLO_SETACTIVELEVEL` (156). Real 2.05 waits for 156 (live-trace finding), so unpatched it stalls on the loading screen. Hence the gs2lib patch, which costs real 2.1 its login.
- `PLI_UPDATEFILE` for files on a hard-coded default list (`idle.gani`, `walk.gani`, `sword?.png`, `body.png`, the stock `.wav` set, `pics1.png`, and others) always answers "up to date" without looking at modtimes. `PLI_WANTFILE` has no such check.
- 2.21 to 2.31 get a hidden clientside weapon, `-gr_zlib_fix`, that pads chat to dodge the 2.x clients' zlib bug (see `../version-profiles/2.22-2.31.md`).
- Listserver link: plain `REGISTERV3` line, then zlib frames: `NEWSERVER`, `SERVERHQPASS`/`SERVERHQLEVEL`, `SETPLYR` + `PLYRADD` per player, `VERIACC2` per login, `SENDTEXT` `Listserver,settings,allowedversions,...`. It never pings on its own.

Real clients against it today [lab]

Loopback namespace, Wine, listserver version 2:

Client	List	Game
2.05 rev 2	14921, shape A (`newmain` only)	in game, `onlinestartlocal.nw`
2.17.1	**14922**, shape B (list name + account, plain zlib); "Classic" tab	in game
2.22	14922, gen 5 (id 7); "Bronze" tab, status, Show More and Donations	in game

2.17.1's choice of 14922 for a plain login is new: the public listserver stays silent to a plain login on 14922 (`../protocol/public-listserver-2026-10-25.md`). So either the 2001-era public server answered plain logins on 14922 too, or 2.17.1 was built against a different list server. Open.

2. GServer-v2 beta4 against our build

`beta4` (`365bbd2`, 2026-10-26) is 807 commits ahead of `efb5b90`: 374 files, +79k/-34k lines. It is a rewrite of most of the tree (`TPlayer → player/PlayerClient*`, `TLevel → level/`), C++23, built with vcpkg, and it fetches gs2lib `5c36ab8` and gs2-parser `73e7ea8` at configure time. **[src]**

What changes for real clients **[src]**:

Area	efb5b90 (ours)	beta4
Classic tokens	1.33 to 1.411, 2.1 up; nothing for 2.00 to 2.05	adds GSERV007 to 018 (1.21 to 1.32), 1.38r1/1.39r1, 2.00 `GNW24011`, 2.01, 2.02, 2.02r2, 2.03, 2.04, **2.05r2 `GNW03081` as `CLVER_2_052`**, NewWorld 2.02 `G2N15041`, Graal Worlds. Still no `G3D15010`.
Level send	separate `sendLevel141` for < 2.1, no `SETACTIVELEVEL`	one path for every version: `LEVELNAME`, board (blank board first on first load), links, signs, chests, `LEVELMODTIME`, then NPCs and **`SETACTIVELEVEL` for every version**
2.05	needs our gs2lib patch, which locks out 2.1	no gs2lib patch needed, but stock beta4 still fails real 2.05 and 2.17.1 (below); with our three-fix patch, in game **[lab]**
zlib bug of 2.x clients	`-gr_zlib_fix` clientside weapon for 2.21 to 2.31	fixed in the receive path: on a zlib data error gs2lib retries as raw DEFLATE and rolls the gen-5 iterator back one step; weapon removed
Big packets	manual `PLO_RAWDATA` handling	gs2lib `CFileQueue` rewritten for RAWDATA (2026-10-26)
Board changes	lost when a level unloads	kept (`def25322`)
Server-side scripts	V8 GS2 NPC server (we build without it)	adds a **GS1 NPC server** (ANTLR grammar), tested on the "newmain" generation; V8 still optional
Listserver link	no keep-alive	`SETIP` every 60 s as keep-alive; `VERIACC2` carries a GSHORT identity length; a `localhost` option skips listserver verification
Ancient clients		bow and bomb power for ancient clients, the bow removed for old clients, loading animation and type-1 tileset fixes

beta4's own notes (`bin/docs/server-unimplemented.md`) say 1.38 to 1.411 and the 2.x/3.x "newmain" era are fully supported client-authoritative, clientside GS1 works up to 5.007, and server-side GS2 is not supported.

beta4 built and run against real clients [lab]

Built here 2026-10-30 (GCC 14, CMake 3.31, system antlr4 4.10 with the Maven Central 4.10.1 jar in place of the pinned 4.13.2 download, `UPNP=OFF`, no V8). Two layers stand between stock beta4 and the old clients.

Config. Stock `allowedversions.txt` uses `[generation-range]` with `classic = GNW13110` (1.41r1 only) and `newmain = GNW03014:GNW28015` (2.22 to 2.31), and the stock generation is `newmain`. So out of the box beta4 refuses 2.05 and 2.17 at login. The lab pack widens `newmain` to `GNW24011:GNW28015` (2.00 to 2.31) and `classic` to 1.33 to 1.41r1; the originals are kept as `*.stock.txt`.

Code. With the versions accepted, the logs show `New login ... [GNW03081 (2.05r2)]` and `[GNW22122 (2.17)]`, and the clients still fail:

- 2.05r2: logs in as `GNW03081 (2.05r2)` and stays on the loading screen.
- 2.17.1: "Invalid PlayerProp" right after login.

Three causes, found by decoding the pcap and comparing beta4's server-to-client order with efb5b90's:

1. Gen-3 clients (2.05, 2.17) put `PLI_LANGUAGE` in the same frame as the login line. beta4's `IPacketHandler::parseLoginPacket` reads the login line and drops the rest of the frame, so the new player's gen-3 iterator is one step behind for the rest of the session. This is the 2.17.1 error.
2. `PlayerClientOriginal` (login type 0) never sends `PLO_SETACTIVELEVEL` (156). The single level path in the table above is `PlayerClientNew`'s; the Original path has no 156. 2.05 waits for it (live-trace finding).
3. `PlayerClientOriginal` sends the first `PLO_LEVELNAME` after the board. efb5b90 and the live-trace stub send it before, and 2.05 stays on loading otherwise.

`../tools/lab-servers/gserver-beta4-lab.patch` fixes all three (keep and replay the login remainder; send 156 from 2.01 up; LEVELNAME first). With it, real 2.05r2, 2.17.1 and 2.22 each list the server on listserver version 2, reach the game and walk, with no unknown-packet logs. One difference from efb5b90 seen in the trace and left alone: beta4 answers a pre-2.1 client's request for a default file with `PLO_FILESENDFAILED` (30) where efb5b90 sent `PLO_FILEUPTODATE` (45); 2.05 did not mind.

None of the three can be done in config. The player class comes from the login type byte (`PlayerLogin::msgLoginPacket`: type 0 is `PlayerClientOriginal`, which 2.05 and 2.17 send), not from `generation` or `allowedversions.txt`; `PlayerClientOriginal`'s level send has no generation or option check; and fix 1 is in the shared login parser. The `generation` setting changes props, baddies, items and NPC details, not these paths.

Checked again with a stock beta4 build (2026-10-30, widened `allowedversions.txt`) because a "legacy port" or old-client toggle would make the patch unnecessary. There is none: beta4 opens one client listener (`serverport`), and its options list has no old-client or encryption switch. The one related knob is `generation`:

- `generation = newmain` (stock): 2.17.1 draws the level, then on the first step the server logs `Unknown Player Packet: 55` and the client shows "Invalid PlayerProp type in constructPropFor" and drops. This is fix 1's iterator desync.
- `generation = classic` with 2.17's token added to the classic range: worse; the login bundle is read as garbage (`Unknown Player Packet: 112`, "sending invalid packets") and the client is dropped before a player exists. beta4's own `docs/start-here.md` warns that 2.x clients on a classic server get mangled packets.

For 1.33 to 1.39 (`generation = classic`, so a second server folder on port 14803) the patch adds three more fixes, and the pack two config changes; details in `../tools/lab-servers/README.md`. The one that matters beyond 1.x: beta4 treats "packet count is 0" as "the next frame is a login" while `PLI_PACKETCOUNT` resets the count, so any client that sends id 31 alone in a frame is misparsed on its next frame and kicked after three. The others: id 25 (1.33 to 1.37 weapon upload) accepted; prop 10 in the bow form on classic; the stock default account's `MAXHP 3.00` loads as 0 hearts (config `MAXHP 3`). Real 1.33 to 1.39 now play on it.

The patch also carries a fourth, smaller fix from the v4 lab: the account loader clears the hidden bit (`STATUS` 0x02), so a player who quit while hiding under a carried object does not log in invisible.

These are bugs against the real clients, not lab choices, so they would be worth upstream; under the project rule nothing was sent outside squirelworks.

3. Listservers

	Our stub (10-22)	**Our v2 (10-30)**	**graal-serverlist `feature/revamp` (2022)**	**Guthius graal-listserver (2025)**
Plain login (id 0 / id 0+1)	yes (list twice for 0+1)	yes, once	yes (needs id 1)	yes, needs `newmain`
Gen 4 (id 4)	yes	yes	yes	no
Gen 5 (id 7, 2.22/2.31)	**no**	yes, two zlib frames as the public 14922	yes	no
RC list (id 5)	no	yes	yes	no
Gserver link	register, VERIACC2	+ players, HQ level tag, drop on disconnect	full (MySQL accounts, profiles, PM, buddies, file serving stubbed out, IRC)	none (static `servers.json`)
Accounts	any	any, or a file	MySQL	JSON files

graal-serverlist is the protocol reference our v2 follows for ids and the HQ-level tags (`U`, `H`, none, `P`, `3`). Its `GETFILE3` file serving is commented out, so no open listserver serves files to a gserver. Guthius's server names the listserver ids by their buttons: 3 "Show More", 5 "pay by credit card" (the 2.22 Donations button), 6 "pay by phone".

4. glauncher and gs2-parser

- **glauncher** (2022) is a Detours DLL plus launcher for the official v4+ client. It hooks `gethostbyname` (maps `<login-host-4x>`, `<graalonline-site>`, `loginserver2/3`) and `connect` (rewrites port 14900) from a line-based `license.graal`. It changes nothing in the protocol. In the lab a bind-mounted hosts file does the same, so we do not need it.
- **gs2-parser** compiles GS2 to the bytecode that v4+ clients and the V8-less beta4 load. GS2 is post-2.x; the 2.x clients this project targets run GS1 text (`../CONSOLIDATED.md`: no GS1 bytecode in 2.05). Useful later only for comparing GS2 bytecode or if Seihai ever runs v4+ worlds.

5. What was changed, and what was not

Changed (lab only, our files):

1. Listserver version 2, with tests. It fixes the missing gen-5 login, the duplicate reply, the stale rows and the dead player count, and adds RC lists, an account file and the site/upgrade URLs. Checked against real 2.05, 2.17.1 and 2.22 and against Seihai 2.00 to 2.31.
2. `run-lab.sh` passes `accounts.txt` to the listserver when it exists.
3. A second gserver pack, `seihai-lab-servers-beta4.tar.gz`: beta4 `365bbd2` with `gserver-beta4-lab.patch`, beside the efb5b90 pack. It runs 2.05 without a gs2lib patch, so 2.1 keeps its token (no 2.1 client was run), the receive-side zlib fix and the GS1 NPC server.

Not changed:

- The efb5b90 pack. It stays the default until other threads have used the beta4 one; both run the same listserver and world.

Real client or invented?

Everything changed today copies a real reply or a behaviour a real client was seen to accept: the gen-5 two-frame reply is the public server's, the plain reply is the one real 2.05 accepted in the 10-21 captures, the tags are what 2.22 draws as tabs. Two things are deliberately lab-only and differ from the public service: our listserver answers

a plain login on 14922 (the public one does not, but real 2.17.1 sends exactly that), and it accepts any account unless an account file is given.

`server-notes/public-allowed-versions.md`

Which client versions public Reborn game servers allow

Date: 2026-10-31. These are the two "Allowed:" lists seen from public game servers, read against gs2lib's version table. No v5 or v6 client has logged in to a public server yet. Everything below about v5/v6 comes from these two lists and the server code.

Tags: **[cap]** seen in a public run's output · **[src]** read in GServer-v2 or gs2lib source · **[inf]** inferred.

The refusals on record

Date	Server	Software	Client sent	Reply
2026-10-29	Graal the Adventure, `<public-host-2gta>:14913`	GServer-v2, listed as "Beta Version: 4.0.0"	Seihai, 2.3 profile (`GNW14015`)	`Your client version is not allowed on this server. / Allowed: 2.22, 5.07`
2026-10-30	"S2" (host not recorded)	gs2, signature 73	Seihai, 2.05 profile (`GNW03081`)	`Your client version is not allowed on this server. Allowed: 2.22, 1.41r1 - iPhone Belote 1.5`

- The 2gta line is from the user's terminal on a personal lab host (hostname redacted), run `debug-20261029-101954`. The same server took the 2.22 profile (`GNW03014`) in every other run that day. **[cap]** The packet logs of that run hold only ids and lengths, so the terminal is the only record of the text.
- The S2 line is from the public game-server check, `public-checks/public-gserver-2026-10-30.md` in the project folder (also `documentation/public-gserver-2026-10-30.md` in Seihai). **[cap]**
- GServer-RS ("S1") closed a 2.05 login with no message, so it gave no list. **[cap]**

How the server builds the list [src]

Read in GServer-v2 `TServer::loadAllowedVersions` and `TPlayer` (the login check), and in gs2lib `5c36ab8 src/IUtil.cpp`:

- Each line of `config/allowedversions.txt` is either a token or a range `A:B`. The message prints each token through gs2lib's version string table, and prints a range as `A - B`.
- A range allows every token whose **position** in `clientVersions[]` lies between A's and B's, both ends included (`versionID >= a && versionID <= b`). A token that is not in the table gets `CLVER_UNKNOWN` (0), which lies in no range.
- The stock `allowedversions.txt` that comes with GServer-v2 enables only `GNW03014` (2.22). Every other line, including all the 5.x and 6.x tokens, is commented out.

What that means for 5.x and 6.x

Graal the Adventure (`2.22, 5.07`): "5.07" is gs2lib's name for `G3D22067`, which is **5.00r7**. That is the only 5.x or 6.x build it allows. 5.12, 5.24, 5.30/5.31 and every 6.x would be refused. **[cap]** for the list, **[src]** for the name.

S2 (`2.22, 1.41r1 - iPhone Belote 1.5`): the range runs from `GNW13110` to `G3D2504D`, which in gs2lib `5c36ab8` covers every 2.x, 3.x, 4.x, 5.x and 6.x entry in the table. **[inf]** The server's own table was not seen, and it refused 2.05 (`GNW03081`), which sits inside that range in `5c36ab8`. So S2's table differs from `5c36ab8`, most likely an older gs2lib with no 2.05 entry. The table below assumes S2's table has the 5.x and 6.x entries in the same order as `5c36ab8`. If it does, S2 should take these builds:

Build	Token	gs2lib name	In S2's range
5.00r7	`G3D22067`	5.07	yes
5.12	`G3D14097`	5.12	yes
5.30r3, 5.31r4, 6.00r2, 6.00r4	`G3D26090`	5.31x	yes
6.01x	`G3D3007A`	6.015	yes
6.034, 6.035	`G3D2505C`	6.034/6.035	yes
6.03r7 Windows / Linux	`G3D0311C` / `G3D0511C`	6.037 / 6.037 (Linux)	yes
5.24r3, 5.24r4, 6.00 beta2	`G3D15010`	not in the table	**no**: unknown tokens are 0
6.00 beta1	`G3D03048`	not in the table	**no**

The listserver does not check the version. The 2026-10-25 runs (`protocol/listserver-login-type-matrix-2026-10-25.md`) show it chooses its reply by login type, not by version or token. Whether a 5.x or 6.x build gets in is up to each game server's `allowedversions.txt`.

Best picks

- **5.00r7** is the only 5.x or 6.x build both servers allow.
- If S2's range is read correctly, **6.03r7** is the 6.x build to try there. It is also the build that played best in the lab (`protocol/v5v6-lab-play.md`).
- **5.24 is refused everywhere** until a server's gs2lib adds `G3D15010` (see `protocol/file-transfer.md`).

Open

- No v5 or v6 login has been made to a public server. A single 5.00r7 or 6.03r7 login to S2 would test the range reading.
- The allowed lists of the other public servers are unknown. A server only prints its list to a client it refuses.

`server-notes/README.md`

Server notes (from decommissioned local workspace)

These files are extracts from the Ember atlas that lived in the local Graal research workspace. They document the server side of the protocol, the lineage of the dumps, and the algorithms used by gs2emu / Ash / Phoenix.

They are provided here so that server-side understanding is versioned alongside the client analysis already in this repository. The full original materials (including ATLAS.html, all Ember markdown, the complete GServerStory, source dumps, Phoenix, and Ash) are in the Google Drive archive linked from `../ARCHIVAL.md`.

Nothing in these notes was invented about the protocol. Identifiers and behavior match the dumps and the Ash rewrite.

Our own servers (2026-10-30)

`lab-servers-2026-10-30.md`: what the project's lab gserver (GServer-v2 `efb5b90` + a gs2lib patch) and listserver are, real 2.05 / 2.17.1 / 2.22 runs against them, and a comparison with GServer-v2 beta4, graal-serverlist, Guthius's graal-listserver, glauncher and gs2-parser.

`setcharprop` and `#n` (2026-10-31)

`setcharprop-hash-n.md`: in a `setcharprop` **value**, `#n` expands to the triggering player's nickname on servers that run GS1 (**[srv]**); property code `#n` sets the NPC's nick. GServer13 has no `setcharprop` (**[bin]**).